

Aniversariantes do Mês

Sistema automatizado de avisos de aniversário de colaboradores via SMS

Nível: 1 — Iniciante

Responsável: Fernando José da Silva Melo

Data do relatório: 11 de setembro de 2026 (atualização — relatório anterior de 03/09/2026)

Status: Implementado, testado (com suíte de testes automatizados) e em produção

Localização do projeto: Aniversariantes do Mes/

1. Objetivo

Construir um sistema que leia automaticamente as datas de nascimento dos colaboradores a partir de um arquivo `NascimentoColaboradores.odt` e envie, via **SMS**, avisos mensais sobre quem faz aniversário no mês corrente, além de lembretes individuais no dia exato de cada aniversário. O sistema roda de forma totalmente autônoma neste computador (Linux), agendado por `systemd timer`.

O sistema originalmente enviava as mensagens via WhatsApp (`whatsapp-web.js`). Em 03/09/2026 foi migrado para SMS — ver seção 5. Em 11/09/2026 foi adicionada uma suíte de testes automatizados e o CI passou a executá-la — ver seção 9.

2. Arquitetura da solução

Camada	Tecnologia escolhida	Motivo
Runtime	Node.js 24	Já disponível no ambiente; <code>node:sqlite</code> e <code>fetch</code> nativos, sem dependências externas para o envio.
Leitura de dados	JSZip + parsing manual de XML	O <code>.odt</code> é um arquivo zip com XML por dentro; não é necessária uma lib pesada de ODF.
Envio de SMS	SMS Gateway for Android (app + relay cloud)	Permite manter o número operador (chip real) como remetente; API HTTP documentada, sem exigir cadastro de negócio como a API oficial da Meta.
Autenticação do envio	HTTP Basic Auth (login/senha gerados pelo app)	Credenciais geradas automaticamente na primeira conexão do app ao modo Cloud; armazenadas em <code>.env</code> (fora do controle de versão).
Persistência	SQLite (<code>node:sqlite</code> nativo)	Nativo do Node 24, sem dependências de compilação; suficiente para controlar idempotência.
Agendamento	<code>systemd timer</code> (usuário)	Mais robusto que <code>cron</code> : log integrado, <code>timezone</code> explícito, <code>retry</code> .
Testes	<code>node --test</code> (nativo)	Sem dependências novas; roda a suíte localmente e no CI. Ver seção 9.

3. Fases implementadas

- Leitura e extração dos dados** — parsing do `.odt` em registros `{nome, dia, mês, ano}`.
- Identificação dos aniversariantes do mês** — filtro por mês corrente, ordenado por dia.
- Geração das mensagens** — mensagem completa (com aniversariantes do mês destacados) e mensagem resumida (ou aviso fixo quando não há aniversariante). Texto plano, sem formatação de negrito (SMS não suporta).
- Envio via SMS** — usando a API do SMS Gateway for Android, chip do número operador como remetente real.
- Agendamento mensal** — verificação todo dia 1º às 09:00.
- Lembretes individuais** — gerados após o envio mensal, disparados no dia exato do aniversário.

4. Decisões técnicas confirmadas

Decisão	Escolha
Mecanismo de envio	SMS Gateway for Android, modo cloud relay (<code>api.sms-gate.app</code>), chip do número operador +55 86 99997-3402
Persistência dos lembretes	SQLite (<code>data/aniversariantes.db</code>)
Agendamento	systemd timer único (<code>diario</code>), 09:00
Formato de destaque	SMS é texto plano — sem negrito. Aniversariante do mês corrente é prefixado com <code>>></code> (antes era <code>*negrito*</code> , padrão WhatsApp)
Fuso horário	<code>America/Fortaleza</code> , explícito no timer e no processo
Falha de envio	Alerta automático enviado por SMS ao número operador

5. Migração WhatsApp → SMS

O sistema foi originalmente construído sobre `whatsapp-web.js` , uma automação não oficial do WhatsApp Web. Em 03/09/2026 foi decidido migrar o envio para SMS puro, mantendo o mesmo número operador (+55 86 99997-3402) como remetente, via o app "SMS Gateway for Android" (open-source, github.com/capcom6/android-sms-gateway), instalado no celular físico do número operador e configurado em modo Cloud Server.

5.1 Credenciais do modo Cloud não são escolhidas pelo usuário

Achado relevante: diferente de um login convencional, as credenciais de autenticação do modo Cloud do SMS Gateway (usuário e senha usados em HTTP Basic Auth) não são definidas pelo usuário — são geradas automaticamente pelo próprio app no celular, na primeira conexão bem-sucedida ao ativar "Cloud Server", e aparecem na tela do app. Uma primeira tentativa usando e-mail/senha de outro serviço resultou em erro `401 Unauthorized` , até identificarmos a documentação oficial confirmando a origem correta das credenciais.

5.2 Campo de mensagem obsoleto na API

A implementação inicial usou o campo `message` no corpo da requisição HTTP, documentado pela própria API como `Deprecated`. Foi corrigido para o formato atual, `textMessage.text` , conforme a documentação oficial (`docs.sms-gate.app`).

5.3 Carregamento de variáveis de ambiente sensível à versão do Node

A primeira versão do carregamento de `.env` dependia da flag nativa `node --env-file-if-exists` (Node 20.12+/21.7+). Como o ambiente tem múltiplas instalações de Node (sistema, nvm), o binário resolvido pelo PATH variava conforme o processo que disparava o script (shell interativo vs. `npm run` vs. `systemd`), causando falhas de "bad option" em alguns casos. Resolvido com um pequeno carregador próprio (`src/loadEnv.js`) que lê o `.env` manualmente via `fs` , independente da versão do Node.

5.4 SMS aceito pela API mas nunca entregue: permissão SEND_SMS bloqueada

Após resolver a autenticação, o envio passava a ser aceito pela API (HTTP 200, estado inicial `Pending`), mas o SMS nunca chegava ao celular de destino. O diagnóstico via endpoint de status da própria API (`GET /3rdparty/v1/messages/{id}`) revelou o erro exato retornado pelo Android:

```
sendSMS: Sending SMS message: uid 10562 does not have android.permission.SEND_SMS.
```

Causa raiz: o app foi instalado via APK fora da Play Store, e desde o Android 13 o sistema bloqueia por padrão a concessão de permissões sensíveis (SMS incluída) para apps de fontes desconhecidas — proteção chamada "Configurações restritas". A permissão só ficou disponível para ativação após o usuário destravar manualmente em Ajustes → Apps → SMS Gateway for Android → menu (:) → "Permitir configurações restritas".

Resolvido: após liberar a permissão `SEND_SMS`, o teste de reenvio confirmou entrega em poucos segundos (estado `Delivered` via API e recebimento confirmado no celular).

Os desafios da fase anterior, baseada em WhatsApp (lock do Chrome, contas com identidade LID, risco de duas execuções simultâneas do navegador), deixaram de se aplicar com a remoção do `whatsapp-web.js` e do Puppeteer/Chrome do fluxo de envio. Esse histórico permanece registrado no `CLAUDE.md` e no histórico do git do projeto.

6. Melhorias de robustez implementadas

Melhoria	Descrição
Timer diário já existente reaproveitado	Nenhuma mudança na estratégia de agendamento — timer único cuida do mensal + lembretes numa só execução por dia.
Carregamento de .env robusto	<code>src/loadEnv.js</code> funciona independente da versão do Node que executa o script (nvm, sistema, systemd, npm).
Erro claro de credenciais ausentes	<code>src/smsClient.js</code> lança mensagem explícita se <code>SMS_GATEWAY_LOGIN</code> / <code>PASSWORD</code> não estiverem definidos, em vez de falhar silenciosamente.
Diagnóstico via status da API	Cada SMS tem um <code>id</code> rastreável; o status (<code>Pending</code> / <code>Delivered</code> / <code>Failed</code>) permite diagnosticar falhas de entrega com o motivo exato retornado pelo Android.
Alerta de falha por SMS	Em caso de erro na execução, o sistema tenta notificar o número operador por SMS antes de encerrar (melhor esforço).
Idempotência preservada	Controle via SQLite (<code>envios_mensais</code> , <code>lembretes.enviado</code>) segue igual à versão anterior — nenhum envio duplicado nos testes.

7. Testes realizados (manuais, ponta a ponta)

Cenário	Comando	Resultado
Geração de mensagens (dry-run)	<code>mensal --dry-run</code>	OK — lógica e formatação em texto plano corretas
Envio mensal real (mensagem completa + do mês)	<code>mensal</code>	OK — ambas com status Delivered confirmado via API e no celular
Lembrete individual no dia exato (simulado)	<code>lembretes</code> (<code>APP_HOJE=2026-09-08</code>)	OK — Delivered confirmado via API e no celular
Idempotência — reexecução no mesmo dia	<code>lembretes</code>	OK — reconheceu envio prévio, não duplicou
Reconstrução do timer de produção	<code>systemctl --user</code> <code>daemon-reload</code>	OK — unit sincronizada com o código atual (SMS)

8. Implantação em produção

- Número de destino final: **+55 86 9986-0339** (`NUMERO_DESTINO=8699860339` em `src/config.js`) — trocado de volta ao valor de produção após validação com o número de teste (o próprio número operador).
- Credenciais do SMS Gateway (`SMS_GATEWAY_LOGIN` / `SMS_GATEWAY_PASSWORD`) configuradas em `.env` , fora do controle de versão (`.gitignore`), com `.env.example` documentando o formato.
- Permissão `SEND_SMS` liberada no celular do número operador (ver seção 5.4).
- Timer `aniversariantes.timer` — já estava instalado em `~/.config/systemd/user/` desde a fase WhatsApp; a unit `.service` foi atualizada para o comando/descrição atuais e recarregada (`daemon-reload`).
- `loginctl enable-linger` já habilitado, garantindo que o timer funcione mesmo sem sessão gráfica ativa.

Status atual: sistema validado ponta a ponta em produção em 03/09/2026 (envio mensal real + lembrete individual, ambos entregues). Próximo disparo agendado do timer: todo dia às 09:00 (horário de Fortaleza); próximo envio mensal completo: 01/10/2026. Acompanhamento via `journalctl --user -u aniversariantes.service` .

9. Testes automatizados e integração contínua

Em 11/09/2026 foi adicionada uma suíte de testes unitários ao projeto, usando o test runner nativo do Node.js (`node --test`) — sem introduzir novas dependências de produção ou de desenvolvimento.

9.1 Cobertura da suíte

Módulo	O que é testado
<code>test/birthdays.test.js</code>	Filtro e ordenação dos aniversariantes do mês e do dia.
<code>test/messages.test.js</code>	Montagem das duas mensagens: destaque <code>>></code> no mês corrente e aviso fixo quando não há aniversariante.
<code>test/parseOdt.test.js</code>	Leitura do <code>.odt</code> , usando um arquivo fictício gerado em memória via JSZip — não depende do <code>NascimentoColaboradores.odt</code> real.
<code>test/db.test.js</code>	Idempotência de envio mensal e geração/baixa de lembretes, usando SQLite em memória — nunca toca em <code>data/aniversariantes.db</code> .
<code>test/smsClient.test.js</code>	Autenticação, conversão para E.164, erro de credenciais ausentes e erro de HTTP, mockando <code>fetch</code> .
<code>test/config.test.js</code>	Função <code>hoje()</code> , incluindo o comportamento de simulação via <code>APP_HOJE</code> .

Ajuste mínimo de suporte: `db.abrirBanco()` passou a aceitar um caminho opcional (mantendo o comportamento padrão inalterado), para permitir isolar os testes do banco de produção real.

9.2 Integração contínua (GitHub Actions)

Achado relevante: o workflow de CI existente rodava em Node 20, versão que não possui o módulo `node:sqlite` (usado por `src/db.js`, que exige Node ≥ 22.5). O CI foi atualizado para Node 24 — a mesma versão que já roda em produção via systemd — e passou a executar `npm test` a cada push/PR para a branch `main`, além do `npm install` já existente.

9.3 Achado colateral: pacote Node órfão na pasta pessoal do usuário

Resolvido: ao investigar por que `npm test` falhava com `Error: No such built-in module: node:sqlite` mesmo com o Node 24 correto instalado, foi identificado um pacote npm órfão (`node@21.6.1 + lts`), instalado desde 30/01/2024 diretamente em `/home/fernandomelo/` — fora de qualquer projeto e sem controle de versão. Esse `node_modules` avulso (203 MB) era priorizado no PATH pelo próprio npm ao rodar `npm run / test / exec` em qualquer subpasta da pasta pessoal, fazendo os scripts executarem com o binário de Node errado. Removido após confirmação do usuário.

9.4 Resultado

Cenário	Comando	Resultado
Suíte completa, local	<code>npm test</code>	OK — 22/22 testes passando (Node 24)
CI (GitHub Actions)	push/PR para <code>main</code>	OK — instala dependências e executa a suíte automaticamente

10. Estrutura de arquivos do projeto

```
Aniversariantes do Mes/
├─ App.md                      # Especificação original do projeto
├─ CLAUDE.md                   # Planejamento, decisões técnicas e status
├─ NascimentoColaboradores.odt # Fonte de dados dos colaboradores
├─ package.json                # Inclui o script "test" (node --test)
├─ .env.example                # Formato das credenciais do SMS Gateway (sem valores reais)
├─ .gitignore                  # Exclui node_modules, .env e o banco
├─ .github/workflows/ci.yml    # CI: Node 24, npm install + npm test
├─ data/
│   └─ aniversariantes.db      # Controle de idempotência (SQLite)
├─ src/
│   ├── config.js              # Número de destino, número operador, data simulável
│   ├── loadEnv.js             # Carrega .env manualmente (independente da versão do Node)
│   ├── parseOdt.js            # Leitura do .odt
│   ├── birthdays.js           # Filtros de aniversariantes
│   ├── messages.js            # Montagem das mensagens (texto plano)
│   ├── db.js                  # Persistência SQLite (abrirBanco aceita caminho custom)
│   ├── smsClient.js           # Envio via API do SMS Gateway for Android
│   └─ index.js                # Comandos: diario | mensal | lembretes
├─ test/                       # Suíte de testes unitários (node --test) – ver seção 9
│   ├── birthdays.test.js
│   ├── messages.test.js
│   ├── parseOdt.test.js
│   ├── db.test.js
│   ├── smsClient.test.js
│   └─ config.test.js
└─ systemd/
    ├── aniversariantes.service
    └─ aniversariantes.timer
```

11. Recomendações futuras

- Monitorar os primeiros envios reais de outubro (dia 1º) pelo `journalctl` para confirmar estabilidade contínua em produção.
- A confiabilidade do envio depende do celular do número operador estar sempre ligado, conectado à internet e com o app aberto/online — é um ponto único de falha. Se isso se mostrar instável ao longo do tempo, avaliar migrar para um provedor de SMS dedicado (ex.: Twilio, Zenvia), com a contrapartida de trocar o número remetente.
- Se a lista de colaboradores crescer muito, avaliar importar o `.odt` para o banco em vez de reler o arquivo a cada execução.
- Reavaliar mensagens muito longas (ex.: a mensagem completa com todos os colaboradores) quanto a custo/segmentação de SMS — mensagens acima de 160 caracteres são quebradas em múltiplos segmentos pela operadora.
- Estender a suíte de testes para os fluxos de orquestração de `src/index.js` (hoje cobertos apenas manualmente, ver seção 7), caso o projeto ganhe complexidade.

